palgrave
macmillan

Digital Signal Processing: Concepts and Applications, second edition covers the basic principles and operation of DSP devices. Its aim is to give the student the essentials of this mathematical subject in a form that can be easily understood and assimilated. The text concentrates on discrete systems, starting from digital filters and discrete Fourier transforms. These are then extended into adaptive filters and spectrum analysers with the minimum of mathematical derivation, concentrating on demonstrating the performance which is achievable from these processors in communications and radar system applications. This new edition has been updated to include learning outcomes and summaries and provide more examples. The text has been completely redesigned and is presented in a clear and easy-to-read style.

Key features:

- Self assessment questions within the text, with answers provided
- Numerous practical worked examples on processor design and performance simulation
- MATLAB® code for animated simulations available to students via World Wide Web access

This textbook is appropriate for undergraduate and MSc courses in signals and systems and signal processing, and for professional engineers who wish to have a simple, easy-to-read reference book on DSP techniques.

Bernard Mulgrew is Professor of Signals and Systems at the University of Edinburgh and leads the Signals and Systems Research Group. He teaches undergraduate classes in systems theory and signal processing and his research interests are in the application of signal processing techniques to radar and communications systems.

Peter Grant is Professor of Electronic Signal Processing at the University of Edinburgh where he is Head of the School of Engineering and Electronics. He teaches digital communications and signal processing and researches in DSP for mobile wireless interface applications.

John Thompson is a lecturer at the University of Edinburgh. He works within the Signals and Systems Research Group, investigating signal processing and antenna array techniques for wireless communications. He teaches undergraduate classes in signal processing and control theory.

ISBN 0-333-96356-3

90101

9 780333 963562

Printed in Great Britain

www.palgrave.com

DIGITAL SIGNAL PROCESSING

Concepts and Applications

Bernard Mulgrew
Peter Grant
John Thompson

Second Edition

visit the website for this book at www.palgrave.com

Thus Φ_{yy} simplifies to:

$$\Phi_{yy} = E\begin{bmatrix} y^2(n) & y(n)\,y(n-1) & y(n)\,y(n-2) \\ y(n)\,y(n-1) & y^2(n) & y(n)\,y(n-1) \\ y(n)\,y(n-2) & y(n)\,y(n-1) & y^2(n) \end{bmatrix}$$

This is a symmetric *Toeplitz* matrix. Thus:

$$\Phi_{yy} = \begin{bmatrix} 1.35 & 0.5 & 0 \\ 0.5 & 1.35 & 0.5 \\ 0 & 0.5 & 1.35 \end{bmatrix}$$

Since the lag d is 1, the z-transform of the cross-correlation sequence is:

$$S_{yx'}(z) = z^{-1}\,(z + 0.5)$$
$$= 1 + 0.5z^{-1}$$

Hence the cross-correlation terms are: $\phi_{yx'}(0) = 1$, $\phi_{yx'}(1) = 0.5$ and $\phi_{yx'}(2) = 0$. The cross-correlation vector is:

$$\Phi_{yx'} = \begin{bmatrix} 1.0 \\ 0.5 \\ 0 \end{bmatrix}$$

Solving the three simultaneous equations yields the impulse response vector of the equaliser:

$$\mathbf{h} = \begin{bmatrix} 0.69 \\ 0.13 \\ -0.05 \end{bmatrix}$$

The minimum MSE can then be obtained using equation (8.9):

$$E[e^2] = \sigma_x^2 - \mathbf{h}^T\,\Phi_{yx'}$$
$$= 0.24$$

It is worth noting that the minimum MSE is larger than the noise variance of 0.1. In attempting to reverse the effect of the channel filter, the equaliser has increased the level of the noise.

Self Assessment Question 8.2: Repeat Example 8.1 for a lag of 2 and show that the MMSE obtained is 0.14.

8.3. Algorithms for adaptive filtering

Conventional signal processing systems, for the extraction of information from a received or observed signal, operate in an open loop manner, i.e. the same processing operation is carried out in the present time interval regardless of whether that operation provided the correct result in the previous time interval. In other words, they make the assumption that the signal degradation is a known and time-invariant quantity (Figure 8.1(a)). The design of optimal linear filters or estimators, such as the Wiener FIR filter of section 8.2, requires more explicit knowledge of the signal environment in the form of correlation functions, state space models or possibly even the probability density functions of Chapter 7. In many situations, such functions are unknown and/or time-varying.

Adaptive processors are designed to approximate these estimators. They operate with a closed loop (feedback) arrangement where the processor frequency response is controlled by the feedback algorithm. This permits them to compensate for time-varying distortions and still achieve performance, close to the optimal estimator function. Adaptive filters use a programmable filter whose frequency response or transfer function is altered, or adapted, to pass without degradation the desired components of a signal and to attenuate the undesired or interfering signals, i.e. to minimise the distortions present on the input signal.

The strategy by which the impulse response of the adaptive filter is altered is the adaptive filter algorithm. An adaptive filter is thus a time-varying filter whose impulse response at a particular time is dependent on the input sequence, the training sequence and the adaptive filter algorithm. The time-varying nature of an adaptive filter gives rise to the concept of convergence. In a stationary environment, the convergence performance is a measure of how many data samples are required for the impulse response of the adaptive filter to approach that of the optimal filter. In a non-stationary environment, the convergence performance is also a measure of how closely the impulse response of the adaptive filter follows the now time-varying impulse response of the optimal filter. Adaptive filters may be both infinite impulse response (IIR) or finite impulse response (FIR) in nature. In this chapter, only adaptive FIR filters will be considered as FIR filters are inherently stable. Thus, provided the feedback loop is properly designed, then stability of the overall adaptive filter will be ensured.

The block diagram of the general adaptive FIR filter is shown in Figure 8.8. The input data sequence, $\{y(n)\}$, is convolved with the FIR sequence, $\{h_i(n)\}$, and the resulting filter output, $\{\hat{x}(n)\}$, equation (6.1), is:

$$\hat{x}(n) = \sum_{i=0}^{N-1} h_i(n-1)\; y(n-i)$$

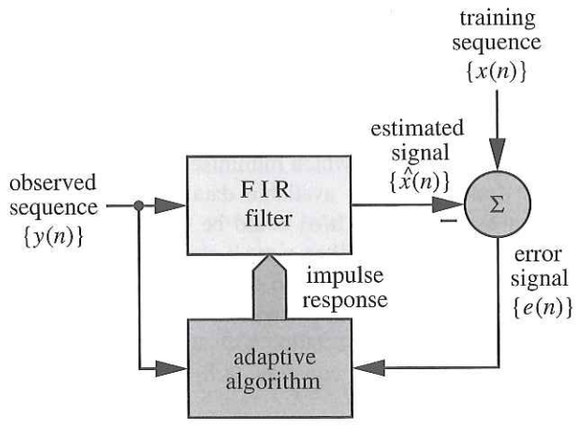


Figure 8.8
General adaptive filter configuration.

This can also be written as a vector inner product:

$$\hat{x}(n) = \mathbf{h}^T(n-1)\ \mathbf{y}(n)$$

The difference here is that the weights of the FIR filter are adapted in a time-varying manner: i.e. $\mathbf{h}(n-1)$ rather than simply $\mathbf{h}$. The current output, $\hat{x}(n)$, is calculated using the *previous* set of weight values, $\mathbf{h}(n-1)$. The training sequence, $x(n)$, is subtracted from the output signal to yield a scalar error signal, $e(n)$, as in equation (8.1):

$$e(n) = x(n) - \hat{x}(n)$$

The error is now used in conjunction with the input signal vector, $\mathbf{y}(n)$, to determine the next *set* of filter weight values, $\mathbf{h}(n)$. The notation $\mathbf{h}(n)$ is used because data up to and including $y(n)$ and $x(n)$ have been used to calculate $\mathbf{h}(n)$. The eventual objective of the adaptive algorithm is to estimate the optimum impulse response. Usually, 'optimum' is defined in an MSE sense, in which case the optimum filter in a stationary environment is the Wiener FIR filter of equation (8.8). The two major classes of adaptive filter control algorithm are now described. These are: recursive least squares (RLS), and the stochastic gradient or least mean squares (LMS) algorithms. Their relative convergence performance and computational complexity will also be covered later, following a discussion of adaptive filter applications.

8.3.1. Recursive least squares

To implement the Wiener FIR filter, as performed previously in section 8.2, both the autocorrelation matrix, $\mathbf{\Phi}_{yy}$, and the cross-correlation vector, $\mathbf{\Phi}_{yx}$, must be obtained from the signal samples. (Strictly speaking, the vector should be typed in lower case bold typefont, to distinguish vectors from matrices, but this is not readily available so the above simpler notation has been adopted here.) The Wiener filter is obtained by minimising the MSE cost function, ξ, which is defined again using the expectation operator, $E[.]$ of equation (8.4). Ensemble averages such as $\mathbf{\Phi}_{yy}$ and $\mathbf{\Phi}_{yx}$ are, however, rarely accessible in practice. Rather, what is available are portions of the data sequences themselves. One method of defining the impulse response of an adaptive filter is to choose the coefficients to minimise a sum of squared errors cost function:

$$\xi = \sum_{k=0}^{n} (x(k) - \hat{x}(k))^2 \tag{8.12}$$

When this cost function is minimised with respect to the impulse response vector, $\mathbf{h}(n)$, associated with the estimate, $\hat{x}(n)$, we obtain a least squares (LS) estimate. The impulse response vector, $\mathbf{h}(n)$, which minimises the sum of squared errors cost function, is now a function of the available data samples rather than ensemble averages. While an expression for $\mathbf{h}(n)$ could be derived by a similar method to that employed to derive the Wiener filter, a more elegant approach is to note that the two optimisation problems are similar and obtain the solution by analogy. Since the sum of squared errors cost function, equation (8.12), was obtained from the MSE cost function by replacing expectation with summation, the impulse response vector $\mathbf{h}(n)$ can be obtained from equation (8.8) by replacing expectation with summation. Thus:

$$\mathbf{R}_{yy}(n)\ \mathbf{h}(n) = \mathbf{r}_{yx}(n) \tag{8.13}$$

where:

$$\mathbf{R}_{yy}(n) = \sum_{k=0}^{n} \mathbf{y}(k)\ \mathbf{y}^T(k) \tag{8.14}$$

and:

$$\mathbf{r}_{yx}(n) = \sum_{k=0}^{n} \mathbf{y}(k)\ x(k) \tag{8.15}$$

In many applications of adaptive filters it is necessary that the impulse response be modified or updated as new data samples appear. The most direct method of meeting this requirement is to increment the upper limits of the summations of equations (8.12), (8.14) and (8.15), and completely resolve equation (8.13). However, a computationally more efficient method is to derive a time recursion for $\mathbf{h}(n)$ in terms of the previous least squares solution $\mathbf{h}(n-1)$ and the new data, $\mathbf{y}(n)$ and $x(n)$. The first step is to obtain recursions for $\mathbf{R}_{yy}(n)$ and $\mathbf{r}_{yx}(n)$. This is straightforward as they are both defined as summations in equations (8.14) and (8.15):

$$\mathbf{R}_{yy}(n) = \mathbf{R}_{yy}(n-1) + \mathbf{y}(n)\ \mathbf{y}^T(n) \tag{8.16}$$

$$\mathbf{r}_{yx}(n) = \mathbf{r}_{yx}(n-1) + \mathbf{y}(n)\ x(n) \tag{8.17}$$

Substitute for $\mathbf{r}_{yx}$ in equation (8.17) using equation (8.13):

$$\mathbf{R}_{yy}(n)\ \mathbf{h}(n) = \mathbf{R}_{yy}(n-1)\ \mathbf{h}(n-1) + \mathbf{y}(n)\ x(n)$$

Then use equation (8.16) to replace $\mathbf{R}_{yy}(n-1)$:

$$\mathbf{R}_{yy}(n)\ \mathbf{h}(n) = \Big(\mathbf{R}_{yy}(n) - \mathbf{y}(n)\ \mathbf{y}^T(n)\Big)\mathbf{h}(n-1) + \mathbf{y}(n)\ x(n)$$

After premultiplying by $\mathbf{R}_{yy}^{-1}(n)$, this yields:

$$\mathbf{h}(n) = \mathbf{h}(n-1) + \mathbf{R}_{yy}^{-1}(n)\ \mathbf{y}(n)\ e(n) \tag{8.18}$$

When the substitution:

$$e(n) = x(n) - \hat{x}(n)$$

is also included:

$$e(n) = x(n) - \mathbf{h}^T(n-1)\ \mathbf{y}(n) \tag{8.19}$$

A recursion for $\mathbf{R}_{yy}^{-1}(n)$ may be obtained by application of the Sherman–Morrison identity to equation (8.16):

$$\mathbf{R}_{yy}^{-1}(n) = \mathbf{R}_{yy}^{-1}(n-1) - \frac{\mathbf{R}_{yy}^{-1}(n-1)\ \mathbf{y}(n)\ \mathbf{y}^T(n)\ \mathbf{R}_{yy}^{-1}(n-1)}{\Big(1 + \mathbf{y}^T(n)\ \mathbf{R}_{yy}^{-1}(n-1)\ \mathbf{y}(n)\Big)} \tag{8.20}$$

Together, equations (8.18), (8.19), and (8.20) are known as the recursive least squares (RLS) algorithm.

Table 8.1 RLS algorithm summary.

Initialise by setting:

$$\mathbf{R}_{yy}(0) = \frac{1}{\delta} I_N, \quad \delta \text{ is a small positive number}$$

$$\mathbf{h}(0) = \mathbf{0}$$

For each time sample $n = 1, 2, \cdots$ do the following:

$$\hat{x}(n) = \mathbf{h}^T(n-1)\,\mathbf{y}(n)$$

$$e(n) = x(n) - \hat{x}(n)$$

$$\mathbf{R}_{yy}^{-1}(n) = \frac{1}{\alpha}\left(\mathbf{R}_{yy}^{-1}(n-1) - \frac{\mathbf{R}_{yy}^{-1}(n-1)\,\mathbf{y}(n)\,\mathbf{y}^T(n)\,\mathbf{R}_{yy}^{-1}(n-1)}{\left(\alpha + \mathbf{y}^T(n)\,\mathbf{R}_{yy}^{-1}(n-1)\,\mathbf{y}(n)\right)}\right)$$

$$\mathbf{h}(n) = \mathbf{h}(n-1) + \mathbf{R}_{yy}^{-1}(n)\,\mathbf{y}(n)\,e(n)$$

The least squares approach, as described above, requires access to all the available data to form the estimate $\hat{x}(n)$. Further, it implicitly assumes that the data are valid over all time, i.e. before time 0, implying that it uses prewindowing, see Chapter 9. In many applications where the underlying processes are non-stationary, it is often more appropriate to minimise the effect of old data by progressively reducing the contribution to the squared error cost function. This is akin to assuming that the processes are stationary over short data records. A convenient technique for providing a 'forgetting mechanism' is to replace the cost function of equation (8.12) by an exponentially weighted cost function:

$$\xi = \sum_{n=0}^{k} (x(n) - \hat{x}(n))^2\, \alpha^{k-n} \tag{8.21}$$

α is a positive constant parameter which is less than or equal to unity. It is often referred to as the *forgetting factor*, as it controls the effective length of the memory of the algorithm. As α is reduced in size, the memory of the algorithm reduces. One of the major attractions of using the exponentially weighted cost function of equation (8.21) is that the resultant RLS algorithm is similar in form and hence complexity to the standard RLS form of equations (8.18), (8.19) and (8.20). The only difference is that equation (8.20) is replaced with:

$$\mathbf{R}_{yy}^{-1}(n) = \frac{1}{\alpha}\left(\mathbf{R}_{yy}^{-1}(n-1) - \frac{\mathbf{R}_{yy}^{-1}(n-1)\,\mathbf{y}(n)\,\mathbf{y}^T(n)\,\mathbf{R}_{yy}^{-1}(n-1)}{\left(\alpha + \mathbf{y}^T(n)\,\mathbf{R}_{yy}^{-1}(n-1)\,\mathbf{y}(n)\right)}\right) \tag{8.22}$$

Table 8.1 summarises the operations involved in the RLS algorithm.

The fact that the initial weight vector is zero is updated into the first set of weight values during the first iteration.

Self Assessment Question 8.3: Verify equation (8.16).

Self Assessment Question 8.4: Verify equation (8.17).

8.3.2. Stochastic gradient methods

The minimum mean-square error (MMSE) definitions of optimality are attractive in that, for an FIR filter, there is only one unique solution and that solution is provided by the Wiener equation (8.8). Many other optimisation problems are less attractive in that direct solutions do not necessarily exist and iterative search techniques must be applied. One of the most important iterative techniques is the method of steepest descent. The importance of this method is that it provides the basis for the stochastic gradient (SG) algorithms which are widely applied in adaptive filtering. An understanding of the steepest descent method can provide a clearer insight into the operation of the stochastic gradient algorithms.

The method of steepest descent is illustrated in Figure 8.9, by examining the contours of constant MSE for a two-coefficient, h_0, h_1, FIR filtering problem. Note in particular the elliptical nature of the contours. In Figure 8.9, the optimum Wiener solution is clearly identified in the centre of the concentric contours which, on a 3-D plot such as Figure 8.6, corresponds to the bottom of the bowl-shaped surface. In the method of steepest descent, an initial guess at the weight vector, $\mathbf{h}_i$, is made. The gradient vector, ∇_i, of the MSE surface for this $\mathbf{h}_i$ value is then calculated as in equation (8.7):

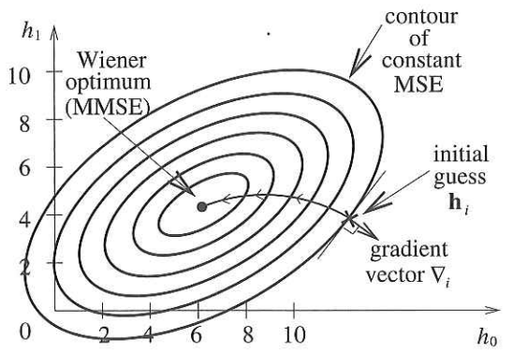


Figure 8.9 Method of steepest descent.

$$\nabla_i = \left[\frac{\partial \xi}{\partial h_0} \; \frac{\partial \xi}{\partial h_1} \cdots \frac{\partial \xi}{\partial h_{N-1}} \right]^T_{\mathbf{h}=\mathbf{h}_i}$$

$$= 2\,\Phi_{yy}\,\mathbf{h}_i - 2\,\Phi_{yx} \tag{8.23}$$

The gradient vector points in the direction of the greatest rate of *increase* of the MSE cost function. The initial guess may thus be improved by moving in the opposite direction, i.e. following the line of steepest descent. Such a move is accomplished by subtracting a scaled version of the gradient, from the initial weight vector, $\mathbf{h}_i$, to give a new vector update, $\mathbf{h}_{i+1}$:

$$\mathbf{h}_{i+1} = \mathbf{h}_i - \mu\,\nabla_i \tag{8.24}$$

The small positive scaling constant μ is usually known as the step size. The optimum solution may be obtained by repeated application of equations (8.23) and (8.24) if the step size lies within a range specified by the largest eigenvalue, $\lambda_{\max}$ (see Appendix A), associated with the autocorrelation matrix, Φ_{yy}:

$$0 < \mu < \frac{1}{\lambda_{\max}} \tag{8.25}$$

The selection of such a magnitude for μ ensures that the weight trajectory always moves towards the MMSE optimum solution (Figure 8.9) and this process is known as convergence of the adaptive filter.

Computation of the gradient of equation (8.23) still requires access to the autocorrelation matrix and cross-correlation vector. Thus the method of steepest descent is not strictly an adaptive filter algorithm but is an indirect method of solving the linear equations summarised by equation (8.8).

To obtain the LMS stochastic gradient algorithm, the gradient of equation (8.23) is replaced by a noisy estimate of the gradient, $\hat{\nabla}$, based on the data, and the iterative step of equation (8.24) is replaced by a time recursion to give:

$$\mathbf{h}(n) = \mathbf{h}(n-1) - \mu\,\hat{\nabla}(n-1) \tag{8.26}$$

The vector $\mathbf{h}(n)$ provides an estimate of the Wiener impulse response at time sample n and the vector $\hat{\nabla}(n)$ is an associated estimate of the gradient. The exact expression for the gradient vector, $\nabla(n)$, in terms of the impulse response vector, $\mathbf{h}(n)$, can be obtained from equation (8.7):

$$\nabla(n) = -2\,E[\mathbf{y}(k)\,(\,x(k) - \mathbf{h}^T(n)\,\mathbf{y}(k))]$$

$$= -2\,E[\mathbf{y}(k)\,e(k)] \tag{8.27}$$

where $e(k)$ represents the error at data sample k of a filter with impulse response $\mathbf{h}(n)$:

$$e(k) = x(k) - \mathbf{h}^T(n)\,\mathbf{y}(k)$$

An estimate, $\hat{\nabla}(n)$, of the gradient, $\nabla(n)$, could be formed by replacing the ensemble average of equation (8.27) by a time-average operation. This would involve holding the filter impulse response constant at $\mathbf{h}(n)$ while data were processed within the filter. However, if the impulse response of the data changes at every sample point as suggested by equation (8.26), the time average reduces to a single multiplication at $k = n$. This leads to simple expressions for the estimated gradient and the associated error:

$$\hat{\nabla}(n-1) = -2\,\mathbf{y}(n)\,e(n) \tag{8.28}$$

$$e(n) = x(n) - \mathbf{h}^T(n-1)\,\mathbf{y}(n) \tag{8.29}$$

Substitution of equation (8.28) into (8.26) yields the LMS algorithm:

$$\mathbf{h}(n) = \mathbf{h}(n-1) + 2\,\mu\,\mathbf{y}(n)\,e(n) \tag{8.30}$$

The block least mean-squares (BLMS) algorithm, where the impulse response of the adaptive filter is held constant for several data samples, represents another possible stochastic gradient technique. It provides a possibly improved estimate of the gradient. This approach is attractive for adaptive filters, particularly when the convolution operation in Figure 8.8 is realised by frequency-domain multiplication, see later (Figure 10.11), as this is also a block processing operation. The BLMS algorithm exploits the further advantage that both the filtering operation and the calculation of the gradient estimate may be implemented very efficiently using the fast Fourier transform (FFT) (see later – Figure 10.11).

An adaptive FIR filter with LMS algorithm is illustrated in Figure 8.10 where the $2\mu e(n)$ operations are clearly seen in the diagram as the updates to the individual $h(n)$ weight values stored in the recursive integrator within the structure. Table 8.2 summarises the operations involved in the LMS algorithm.

Table 8.2 LMS algorithm.

Initialise by setting:

$$\mathbf{h}(0) = \mathbf{0}$$

For each time sample $n = 1, 2, \cdots$ do the following:

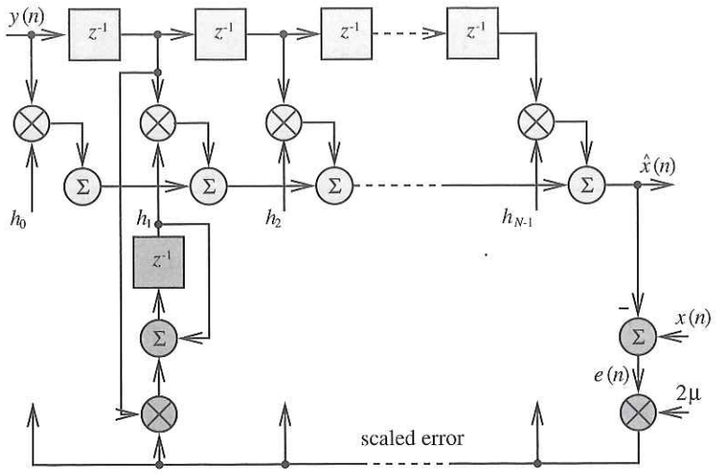
$$\hat{x}(n) = \mathbf{h}^T(n-1)\,\mathbf{y}(n)$$

$$e(n) = x(n) - \hat{x}(n)$$

$$\mathbf{h}(n) = \mathbf{h}(n-1) + 2\,\mu\,\mathbf{y}(n)\,e(n)$$

The simplest method of performing a theoretical analysis of the convergence of adaptive filter algorithms considers the evolution of the mean of the impulse response vector of the adaptive filter and attempts to show that, as the time index n increases, the mean will tend to the impulse response of the Wiener filter. This technique, first suggested by Widrow for the LMS algorithm, while not being strictly rigorous, does clearly highlight the major factors which control the convergence rate of the SG algorithms. Simplifications such as this are essential as convergence analysis is a very complicated task requiring considerable mathematical sophistication.

It can be shown that the convergence performance is limited by the modes of the filter which are in turn related to the eigenvalues, λ_i where $i = 1, \cdots, N-1$, of the input autocorrelation matrix, Φ_{yy}. A time constant, τ_j, can be defined for mode j and is given approximately as:



gure 8.10
he least mean-square
MS) adaptive filter
esign.

$$\tau_j \approx \frac{1}{2\mu\lambda_j}$$

where λ_j is the eigenvalue associated with that mode. The largest time constant, $\tau_{\max}$, is due to the smallest eigenvalue, $\lambda_{\min}$, and is thus given by:

$$\tau_{\max} \approx \frac{1}{2\mu\lambda_{\min}} \tag{8.31}$$

Combining equation (8.25) with the limiting case in equation (8.31) gives:

$$\tau_{\max} > \frac{\lambda_{\max}}{2\lambda_{\min}} \tag{8.32}$$

which suggests that the larger the eigenvalue ratio (EVR) or condition number, $\lambda_{\max}/\lambda_{\min}$, of Φ_{yy}, the longer the LMS algorithm will take to converge. The eigenvalues of the input autocorrelation matrix can be related to the power spectral components in the signal. Thus strong signal spectral components result in high loop gain in the adaptive filter and have an associated small τ value. The weak signal spectral components give rise to the $\tau_{\max}$ value.

Insight into the relationship between EVR and the convergence performance of the LMS algorithm can be gained through a consideration of the steepest descent algorithm of equation (8.24). Figure 8.11(a) illustrates how the eigenvalues ($\lambda_{\max}$, $\lambda_{\min}$) and their associated eigenvectors ($\mathbf{v}_{\max}$, $\mathbf{v}_{\min}$) of the autocorrelation matrix Φ_{yy} define the contours of constant MSE for a two-tap adaptive filtering problem. In Figures 8.11(b) and 8.11(c), the operation of the steepest descent algorithm is illustrated for EVRs of 2 and 4 respectively. In the latter two figures, an asterisk indicates the optimum solution. The eigenvectors of the (2×2) autocorrelation matrix Φ_{yy} are in the direction of the principal axes of the ellipses of constant MSE. The EVR is simply the ratio of the minor axis to the major axis. Since the eigenvectors are identical in Figures 8.11(b) and 8.11(c), the orientation of the ellipses in the two figures is identical. In both cases, 10 steps of the steepest descent algorithm are taken from an initial starting point of $\mathbf{h} = [4\ 5]^T$. For an EVR of 2, the algorithm is closer to the optimum solution after 10 steps than for an EVR of 4. For the higher EVR case, the contours of constant MSE are closely

approximated by parallel lines in the region of the starting point. The algorithm moves initially on a trajectory parallel to $\mathbf{v}_{\max}$ and only much later turns towards the optimum solution compared with the case where the EVR is 2.

In the practical situation it is difficult or impossible to determine the eigenvalue ratio and hence find an optimum value for the stepsize μ. A more practical limit is given by:

$$0 < \mu < \frac{1}{3N\sigma_y^2} \tag{8.33}$$

where N is the number of taps and $\sigma_y^2 = E[y^2(k)]$ represents the variance of the input signal. It is often possible to produce a crude estimate of σ_y^2 from knowledge of the application. Often a reasonable choice for μ is to set it to the mid-point of the region, i.e.:

Figure 8.11
Eigenvectors, eigenvalues and convergence:
(a) the relationship between eigenvectors, eigenvalues and constant MSE contours;
(b) steepest descent for EVR of 2;
(c) EVR of 4.

$$\mu = \frac{1}{6N\sigma_y^2} \tag{8.34}$$

Self Assessment Question 8.5: An adaptive filter has a white noise input of variance 2. The cross-correlation vector Φ_{yx} has a value $[3\ 1]^T$. Calculate the gradient when the impulse response vector $\mathbf{h}$ of the filter is $[2.9\ \ 4.0]$.

EXAMPLE 8.2

LMS algorithm

The table below shows the input and training signal to an adaptive filter at sample numbers 4 and 5:

k	$y(k)$	$\mathbf{h}(k)$	$x(k)$
4	0.25	$[1.2\ 3.7]^T$	1.03
5	0.5		−0.27

Using the LMS algorithm, evaluate $\mathbf{h}(5)$. The step size is fixed at 0.01.

Solution

Using equation (8.29):

$$e(5) = x(5) - \mathbf{h}^T(4)\ \mathbf{y}(5)$$

$$= -0.27 - [1.2\ 3.7]\begin{bmatrix}0.5\\0.25\end{bmatrix} = -1.795$$

The impulse response is updated using (8.30):

$$\mathbf{h}(5) = \mathbf{h}(4) + 2\mu\ \mathbf{y}(5)\ e(5)$$

$$= \begin{bmatrix}1.2\\3.7\end{bmatrix} + 2(0.01)\begin{bmatrix}0.5\\0.25\end{bmatrix}(-1.795)$$

$$= \begin{bmatrix}1.1821\\3.6910\end{bmatrix}$$

EXAMPLE 8.3

RLS algorithm

Using the table of Example 8.2, evaluate $\mathbf{R}_{yy}^{-1}(5)$ given that:

$$\mathbf{R}_{yy}^{-1}(4) = \begin{bmatrix}1.26 & -0.13\\-0.13 & 1.26\end{bmatrix}$$

Solution

Rewriting equation (8.20) to emphasise the order in which the calculation is performed:

$$\mathbf{R}_{yy}^{-1}(5) = \mathbf{R}_{yy}^{-1}(4) - \frac{\left(\mathbf{R}_{yy}^{-1}(4)\ \mathbf{y}(5)\right)\left(\mathbf{R}_{yy}^{-1}(4)\ \mathbf{y}(5)\right)^T}{1 + \mathbf{y}^T(5)\left(\mathbf{R}_{yy}^{-1}(4)\ \mathbf{y}(5)\right)}$$

Note that $\mathbf{R}_{yy}$ is symmetric and hence $\mathbf{R}_{yy}^{-1}$ is symmetric. Therefore $(\mathbf{R}_{yy}^{-1})^T = \mathbf{R}_{yy}^{-1}$. Thus in using (8.20) it is more economical to calculate an intermediate variable vector $\mathbf{u}(5)$ where:

$$\mathbf{u}(5) = \mathbf{R}_{yy}^{-1}(4)\ \mathbf{y}(5)$$

$$= \begin{bmatrix}1.26 & -0.13\\-0.13 & 1.26\end{bmatrix}\begin{bmatrix}0.5\\0.25\end{bmatrix} = \begin{bmatrix}0.5975\\0.25\end{bmatrix}$$

Thus:

$$\mathbf{R}_{yy}^{-1}(5) = \begin{bmatrix}1.26 & -0.13\\-0.13 & 1.26\end{bmatrix} - \frac{\begin{bmatrix}0.5975\\0.25\end{bmatrix}[0.5975\ 0.25]}{1 + [0.5\ 0.25]\begin{bmatrix}0.5975\\0.25\end{bmatrix}}$$

$$= \begin{bmatrix}1.26 & -0.13\\-0.13 & 1.26\end{bmatrix} - \begin{bmatrix}0.3570 & 0.1494\\0.1494 & 0.0625\end{bmatrix}\frac{1}{1.3613}$$

$$= \begin{bmatrix}0.9978 & -0.2397\\-0.2397 & 1.2141\end{bmatrix}$$

8.3.3. A comparison of algorithms

Simulation results are presented here to illustrate the differences in performance between the two major algorithm types that have been discussed in the previous sections. The structure used to highlight these differences is illustrated in Figure 8.12. This is an example of a system modelling mode of application where the objective is to use the adaptive filter to identify the impulse response $\mathbf{h}_{opt}$ of an unknown system from input and output data. The noise shaping filter is a useful mechanism for adjusting the characteristic of (conditioning) the input signal to the unknown system and the adaptive filter, $y(n)$, and hence to control the eigenvalue ratio. The error vector norm or deviation from the optimum weight vector, $\mathbf{h}_{opt}$, is defined as:

$$\rho(n) = E[(\mathbf{h}(n) - \mathbf{h}_{opt})^T\ (\mathbf{h}(n) - \mathbf{h}_{opt})] \tag{8.35}$$

This was chosen as the preferred measure of performance over the output MSE.

The error vector norm permits the convergence plots to be plotted to demonstrate clearly the operation of the algorithm below the output noise floor, which is determined by the noise added to the training signal.

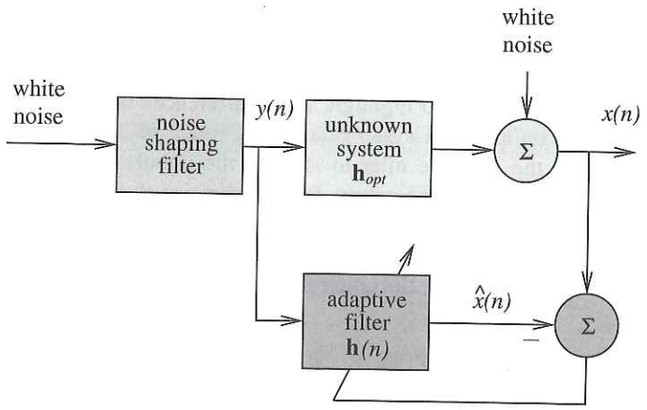
Three sets of results for a $N = 16$-coefficient adaptive FIR filter are illustrated in Figure 8.13. For all simulations, the additive noise was set at –60 dB. Firstly we consider the initial performance of the various algorithms under good SNR conditions. When the input signal is white (Figure 8.13(a)), the condition number or eigenvalue ratio of the input autocorrelation matrix is unity. The RLS algorithm then converges to the noise floor in approximately 30 or $2N$ data samples or iterations. The LMS algorithm reaches the noise floor within approximately 400 iterations.

If the eigenvalue ratio of the input signal is increased to 11, by altering the frequency response of the noise shaping filter, the performance of the RLS algorithm, depicted in Figure 8.13(b), is hardly altered. On the other hand, the performance of the LMS algorithm is significantly degraded, now taking greater than 1000 iterations to reach the noise floor. Finally, in Figure 8.13(c) the eigenvalue ratio is 68, which would be considered to be extremely severe conditions. The RLS reaches the noise floor in 70 iterations while the LMS now takes approximately 2000 iterations.

When the performance goal is to achieve the noise floor level, the results can be summarised as follows. The RLS provides consistent fast convergence independent of the eigenvalue ratio. The LMS, on the other hand, provides the poorest performance, being highly dependent on the eigenvalue ratio. The second point of interest is the performance of the algorithms below the noise floor. For instance, in all three cases the convergence rate of the RLS algorithm decreases dramatically after it reaches the noise floor. However, the LMS algorithm converges to a constant level on or just below the noise floor. This is a consequence of the choice of step size, μ, that was employed. The LMS step size for all these simulations was set at $1/(6N)$ in order to ensure convergence over the wide range of input conditions. The final value of the norm achieved by the LMS algorithm could have been improved if a smaller step size had been used, but this would have resulted in a further slowing down of the overall rate of convergence.

The choice of an appropriate adaptive algorithm for a particular application involves an assessment of the computational complexity as well as the convergence performance. Considerable research effort has been applied to achieve

re 8.12
em identification
ation.



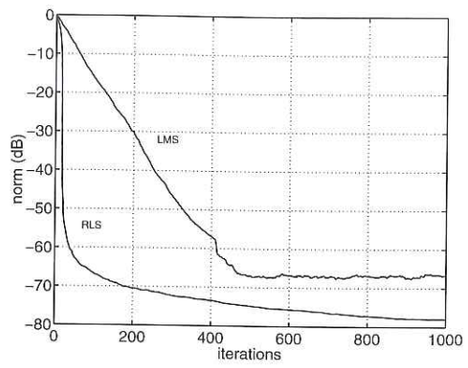
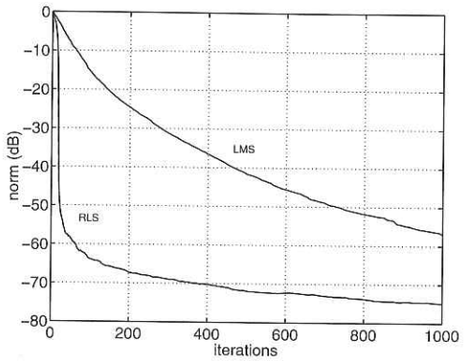
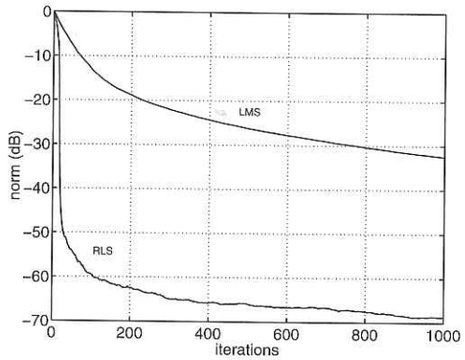


Figure 8.13
Convergence plots for a 16-tap adaptive filter performing system identification:
(a) shows results with an eigenvalue ratio (EVR) of 1, i.e. a white noise signal;



(b) shows the EVR increased to 11; and



(c) 68 by progressively increasing the spectral coloration of the input signal.

computationally efficient implementations of the various adaptive filter algorithms. Some of these are summarised in Table 8.3, where their complexity is compared in terms of computational multiplication, division and add/subtraction operations for a filter of order N. Here the RLS algorithm is based on a Kalman filter [Bozic]. Note that this also needs a division operation which presents severe practical problems for high speed implementations. Table 8.3 also includes the block LMS approach where the filter operation is performed by frequency-domain processing, see later (Figure 10.11), rather than the FIR time-domain convolution operation of Figure 6.1. The BLMS is particularly significant as the computations scale by $\log N$ rather than N and, in high-complexity systems, with long FIR filters this

approach will be the preferred solution.

Table 8.3 Complexity comparison of N-point FIR filter algorithms.

Algorithm class	*Implementation*	*Computational load*		
		multiplications	*adds/subtractions*	*divisions*
RLS	fast Kalman	$10N+1$	$9N+1$	2
SG	LMS	$2N$	$2N$	–
	BLMS (via FFT)	$10\log(N)+8$	$15\log(N)+30$	–

Self Assessment Question 8.6: Calculate the number of multiplications associated with a 4-tap and 128-tap adaptive filter trained with the LMS algorithm; repeat the calculation for an FFT-based BLMS algorithm. What conclusions do you draw?

Self Assessment Question 8.7: What are the relative advantages and disadvantages of the RLS and LMS algorithms?

8.4. Applications

There are basically three modes of operation of an adaptive filter and these are illustrated in Figure 8.14:

(i) Direct system modelling (Figure 8.14(a)) which is typified by the application to echo cancellation where the echoes in the unknown system are duplicated in the adaptive filter and then cancelled in the summer.

(ii) Inverse system modelling (Figure 8.14(b)) which is what is normally implied in the communications channel equalisation application (section 8.4.5), to overcome signal distortion and bandlimiting in the transmission channel.

(iii) Prediction (Figure 8.14(c)), for instance as used in autoregressive spectral analysis and in the linear predictive coding (LPC) of speech (Figure 9.21).

Here we examine how these modes are reflected in the practical application of adaptive filters. The initial question that most people ask is, 'where does the training signal come from?' Hopefully this is answered for each of the following applications. The applications cover: adaptive line enhancement; adaptive tone suppression; noise whitening; echo cancellation; and channel equalisation.

8.4.1. Adaptive line enhancement

Consider the situation illustrated in Figure 8.15(a). We have a narrow-band signal of interest centred at a frequency f_0. This signal might be a single sine wave of frequency f_0 or an amplitude modulated carrier. The signal is contaminated by adding broad-band random noise to it – the noise might be thermal in nature and generated by the amplifiers in a communications receiver. If we knew beforehand (*a priori*) both the bandwidth (B) of the signal and the centre frequency, we could design an analogue or digital bandpass filter with the same centre frequency and

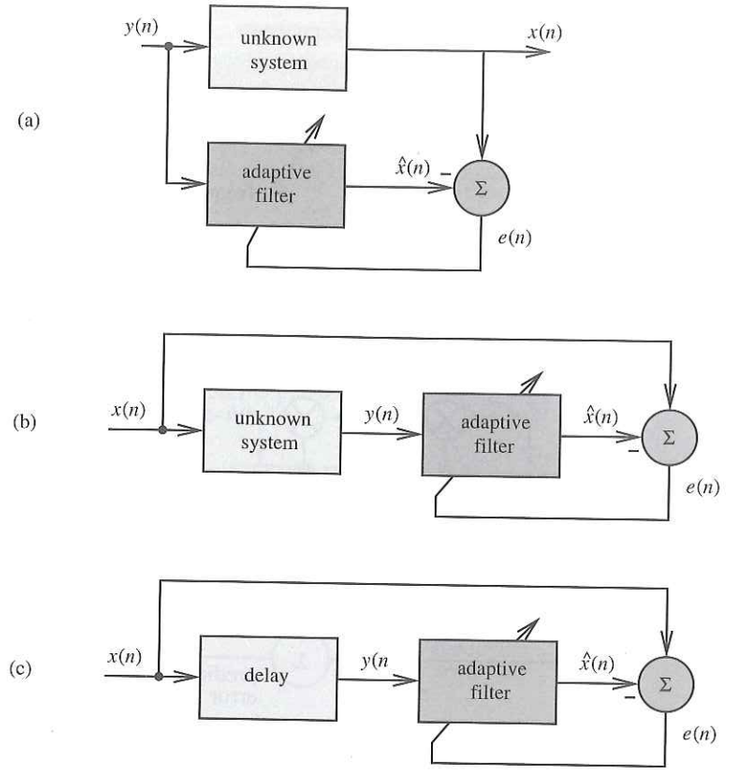


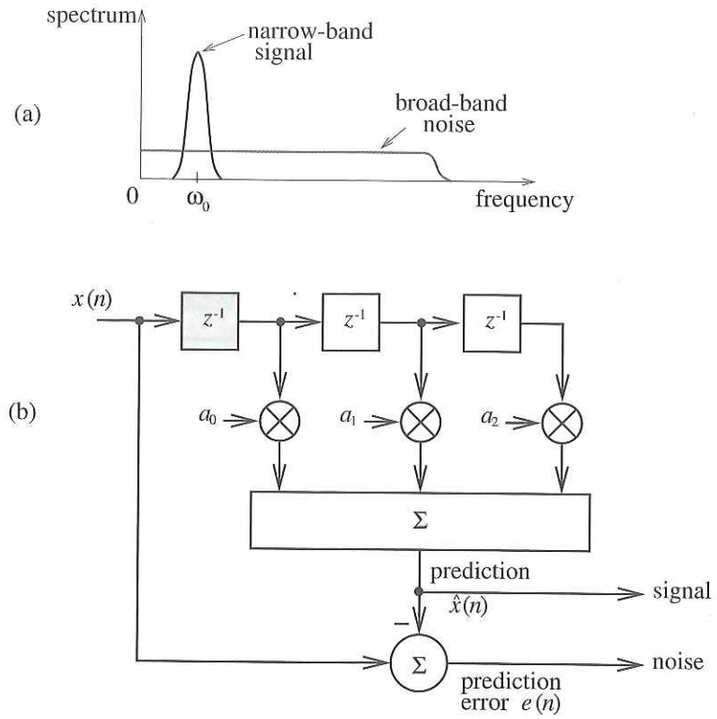
Figure 8.14 Modes of operation of the adaptive filter: (a) direct system modelling; (b) inverse system modelling; (c) linear prediction.

bandwidth. This filter would attenuate the noise components at frequencies outside its bandwidth and hence the signal-to-noise ratio at the output of the filter would be better than the signal-to-noise ratio at the input – we would have cleaned up the signal.

If, however, we only knew that the signal was narrow-band and did not know either its centre frequency or its bandwidth, we could not design such a filter. In cases such as this, an adaptive filter solution may be appropriate. A narrow-band signal such as a sine wave is by its very nature predictable. If we were given 10 consecutive samples of a sine wave we could make a good guess at what the next sample is likely to be. A sine wave is a highly correlated signal. Broad-band noise, on the other hand, is highly uncorrelated from sample to sample. If we were given 10 samples of broad-band noise, we would have very little idea what the next sample would be. It is this difference in predictability between the narrow-band signal and the broad-band noise that we can exploit to separate the signal from the noise.

Figure 8.15(b) illustrates a 3-tap predictor to which we apply $x(n)$ which contains both the signal and the noise. $x(n)$ is delayed by one sample and we use older samples such as $x(n-1)$, $x(n-2)$, $x(n-3)$ to estimate the current input $x(n)$. The estimate $\hat{x}(n)$ is thus a prediction of $x(n)$ based on the samples $x(n-1)$, $x(n-2)$, $x(n-3)$. Because the narrow-band signal is more 'predictable' than the broad-band noise, the prediction $\hat{x}(n)$ will contain less noise than $x(n)$.

To train the predictor we want to make $\hat{x}(n)$ as close an approximation to $x(n)$ as possible. To do this we could use either of the adaptive filtering algorithms



ure 8.15
otive line enhancer:
ignal spectrum;
nhancer configuration.

presented earlier in this chapter. The training signal is $x(n)$, while the input to the adaptive filter is $x(n-1)$ which is equivalent to $y(n)$ in the general adaptive filter structure of Figure 8.8. When the adaptive filter has been trained, $\hat{x}(n)$ will contain mostly narrow-band signal and the error $e(n)$ by implication will contain mostly the broad-band noise. In this signal enhancement application, the output of interest is $\hat{x}(n)$ as shown in Figure 8.15(b). In summary, only one input signal is required. It can be used to provide both the input and the training signal to the adaptive filter!

The filter in Figure 8.15(b) with input $x(n-1)$ and output $\hat{x}(n)$ is the prediction filter and it has transfer function: $a_0 + a_1 z^{-1} + a_2 z^{-2}$. In the above example we would expect the frequency response of this filter to be bandpass with a centre frequency ω_0 – after training of course. Thus $\hat{x}(n)$ is the required output for the earlier adaptive line enhancement application.

The filter with input $x(n)$ and error output $e(n)$ is the prediction error filter and has transfer function: $1 - a_0 z^{-1} - a_1 z^{-2} - a_2 z^{-3}$. In the above example we would expect the frequency response of this filter to be bandstop with the notch centred at angular frequency ω_0 – after training.

8.4.2. Adaptive tone suppression

Another very similar application of adaptive filtering in a prediction mode is illustrated in Figure 8.16. In this example the signal of interest is broad band in nature – it might be the signal transmitted by a wide-band communications system such as spread spectrum. This time the interference is narrow band in nature – it might be coming from a large rotating machine or fan in the same room as the

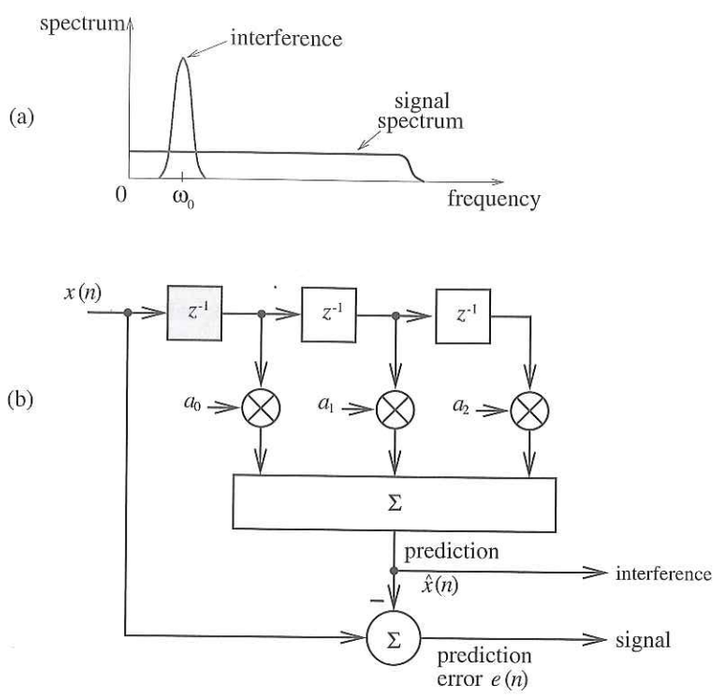


Figure 8.16
Prediction error filter:
(a) signal spectrum;
(b) predictor configuration.

receiver. The adaptive filter solution is almost identical to the previous example except that we are interested in 'cleaning up' the unpredictable part – the broad-band signal. Thus the estimate $\hat{x}(n)$ contains mostly narrow-band interference and the error $e(n)$ from the prediction error filter contains the recovered cleaned-up signal providing the overall output of the system.

8.4.3. Noise whitening

Figure 8.17(a) illustrates the power spectral density of a random signal. The spectrum is not flat and hence the random signal is coloured or correlated. In applications such as linear predictive coding (LPC) of speech or autoregressive (AR) spectral analysis (Chapter 9), we need a filter which will whiten such a signal. If we apply the correlated random signal to the whitening filter, the output will have a white spectrum. The design of the whitening filter is obviously dependent on the signal so an adaptive solution can again be used.

Figure 8.17(b) illustrates the prediction structure. When the adaptive filter has been trained, $\hat{x}(n)$ is the predictable part of $x(n)$. Thus $e(n)$ is the unpredictable part and must have an approximately white spectrum. The prediction error filter is thus the required whitening filter.

Self Assessment Question 8.8: White noise is applied to a bandpass filter. The bandwidth of the filter is decreased at the beginning of each day. Is the signal at the output of the filter more or less predictable at the end of the week?

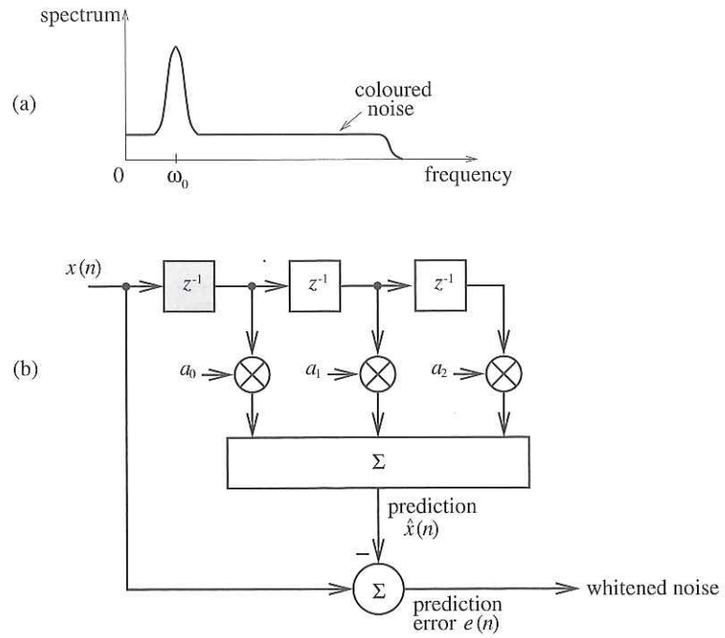


Figure 8.17
Noise whitening filter: (a) input spectrum; (b) whitener structure.

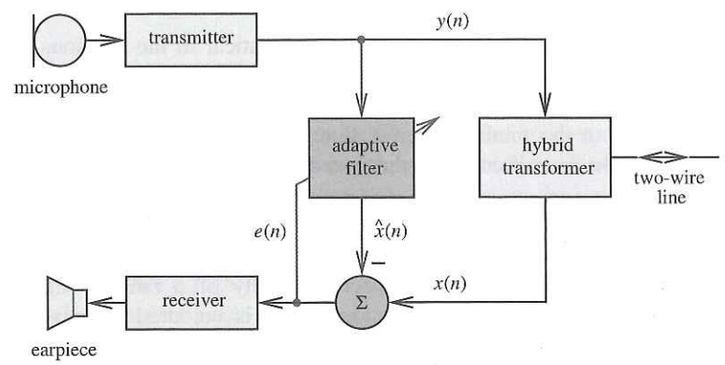


Figure 8.18
Application of adaptive echo cancellation in a telephone handset.

8.4.4. Echo cancellation

The essential elements of a telephone handset are illustrated in Figure 8.18. We speak into the microphone which, in combination with the transmitter, converts the sound energy to electrical energy to give the signal $y(n)$. This signal is applied to a hybrid transformer which allows most of the energy to pass up the two-wire line to the local exchange with the subsequent connection to the person at the other end. The hybrid transformer is not perfect and some of the transmitted energy leaks across it into the receiver (ignore the adaptive filter for the moment). We can view the transformer as a filter with a frequency response or transfer function relating its input $y(n)$ to its output $x(n)$.

While we are speaking into the handset we can hear a slight 'echo' of our own voice in the ear piece. For speech this is a good thing since with this 'feedback' we

can adjust the level of our voice appropriately. When someone is speaking to us from the far end, the signal travels up the two-wire line from the exchange and is routed by the hybrid transformer into the receiver and our ear piece. The receiver amplifier is quite sensitive because it has to amplify the signal after the signal has passed all the way up the line from the exchange.

Unfortunately, when we wish to use the handset as a modem for transmitting binary data, the 'echo' across the hybrid becomes a major problem. The binary data we transmit through the mouth piece transmitter leak across the hybrid and corrupt the received data coming from the exchange.

If we knew the impulse response or transfer function of the hybrid we could build a filter with similar characteristics and apply the signal $y(n)$ to its input. The output of this filter $\hat{x}(n)$ would be a good estimate of the 'echo' component of the signal coming from the hybrid. If we then subtract $\hat{x}(n)$ from $x(n)$, the resultant signal $e(n)$ would have no component of the 'echo' in it. Hybrid transformers are manufactured on a production line. Their characteristics vary widely from one to the other because the manufacturer wants to keep the price of a handset as low as possible. When we wish to use a particular handset with our modem, we have no idea what the impulse response of the hybrid transformer will be – thus we need an adaptive solution.

Figure 8.18 illustrates an adaptive echo canceller for use with a modem. It is an adaptive filter in a system identification mode of operation. When a modem is switched on, it goes through a long initialisation procedure with the exchange. During part of the initialisation, the exchange does not transmit to the handset and the handset transmits a random binary sequence. This random binary sequence forms the input to both the adaptive filter and the hybrid. The output from the hybrid is the training sequence for the adaptive algorithm. As more random data are passed through the hybrid, the adaptive filter learns its impulse response. After the adaptive filter has reduced the MSE to a suitable level, the random binary sequence is stopped, the adaptive algorithm stops updating the weights and the modem is ready to send and receive data.

8.4.5. Channel equalisation

If we transmit a binary data sequence over a radio communications channel or a telephone channel, the signal can be very severely distorted. Looking at the signal at the output of the A/D converter in the receiver we would have no idea what digital data had been transmitted.

A communications channel including: transmitter filter; multipath effects; and receive and filter – can be accurately modelled as a finite impulse response filter (Figure 6.1). Usually the transmitted data look like a random binary sequence where $x(n)$ is either +1 or −1 depending on whether we wish to transmit 'logic one' or 'logic zero'. Gaussian white noise (receiver amplifier noise) is added to the output of the FIR filter to produce the received samples. In its simplest case all the coefficients of the FIR filter would be zero except h_0. The received signal samples would then be:

$$y(n) = \pm h_0 + \text{noise}$$

and we could tell what data were being transmitted by simply testing whether $y(n)$ was greater than or less than zero, i.e.:

if $y(n) \geq 0$ then $x(n) = +1$ else $x(n) = -1$

If, however, the other coefficients were not zero we could not use such a simple test, e.g.:

$$y(n) = \sum_{i=0}^{2} h_i \, x(n-i) + \text{noise} \tag{8.36}$$

Here the received sample $y(n)$ is a function of 3 transmitter bits (or symbols). This is called intersymbol interference (ISI).

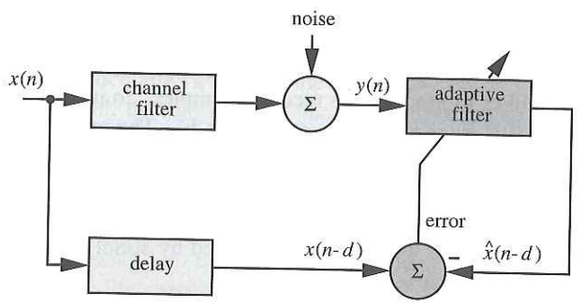
Each channel has a particular frequency response. If we knew this frequency response we could build a filter in the receiver with the 'opposite' or inverse frequency response, as in Figure 7.15. Everywhere the channel had a peak in its frequency response, the inverse filter would have a trough and vice versa. The frequency response of the channel in cascade with the inverse filter would have a flat amplitude response and a linear phase response, i.e. as far as the data were concerned, the cascade of the channel and the inverse filter together would look like a simple delay. Effectively we would have 'equalised' the frequency response characteristics of the channel.

In practice, when we switch on our digital mobile radio or use a modem to transmit data down a telephone line we have no idea what the frequency response of the channel between the transmitter and the receiver will be. What is worse is that every phone call will produce a new channel. We again need an adaptive solution.

Figure 8.19 illustrates an adaptive equalisation scheme. There is a transversal filter or FIR filter in the receiver which we wish to train adaptively to form the inverse to the channel filter. The problem is finding a training signal. Fortunately, we design the transmitter as well as the receiver. When the transmitter is switched on, it sends a random binary sequence or training sequence before the actual data are sent. This binary sequence can be stored in the receiver in memory and used as a training signal for the adaptive filter. The objective is to make the output $\hat{x}(n)$ as close to the transmitted data as possible, in which case the adaptive algorithm would have 'learned' the impulse response of the inverse filter. We can then switch off the adaptive algorithm and start transmitting actual data traffic.

On telephone lines the channel filter does not change with time. So once we have trained the equaliser, we can switch off the adaptive algorithm until we dial up a new line. In digital mobile radio this is not the case. The transmitter may be in a car and, as the car moves, the channel between the transmitter and receiver will change with time. Thus the optimum inverse filter needs to change with time. The receiver needs to track changes in the channel. If the channel is changing while

re 8.19
adaptive equaliser.



actual data are being transmitted, where do we get a training signal from?

One way of doing this is to use what is called 'decision directed' operation (Figure 8.20). If the equaliser is working well then $\hat{x}(n)$ will just be the binary data plus noise. We can remove the noise in a decision circuit which simply tests whether $\hat{x}(n)$ is greater or less than zero. So if the equaliser is working well, the output of the decision circuit is the transmitted data. We could use such data to continue training the adaptive filter by putting the switch in the upper position (Figure 8.20). Here the error is the difference between the binary data and the filter output. This 'decision directed' system will work well, provided the equaliser continues to make correct decisions on the received data samples. If it makes an incorrect decision the training signal will be incorrect, and the adaptive filter will learn the wrong inverse filter, which will lead to more incorrect decisions, i.e. error propagation will result. The decision directed equaliser thus works well in slowly changing channels where the adaptive filter feedback loop can track these changes. If the filter cannot track the signal, then the switch in Figure 8.20 must be moved to the lower position and the actual transmitted data used again to train the filter, as previously shown in Figure 8.19.

CHAPTER SUMMARY

The Wiener filter is optimum in a mean-square error (MSE) sense, i.e. it is the filter, of all possible linear filters, which provides the smallest mean-square error. If the filter is restricted to an N-tap FIR filter, the optimum choice is found by setting the gradient of the MSE (with respect to the filter impulse response vector) to zero. This leads to the Wiener equation which identifies the optimum filter as the solution to a set of N linear simultaneous equations. These equations are defined in terms of the autocorrelation matrix Φ_{yy} associated with the vector $\mathbf{y}(n)$ of signal samples in the filter, and the cross-correlation vector Φ_{yx} associated with the vector $\mathbf{y}(n)$ and the desired scalar output of the filter $x(n)$.

The design of the Wiener filter requires knowledge of the associated autocorrelation and cross-correlation functions, and is rarely used in practical applications. However, the importance of the Wiener FIR filter lies in its role in the design of adaptive FIR filters where it can be used as a benchmark against which performance can be measured.

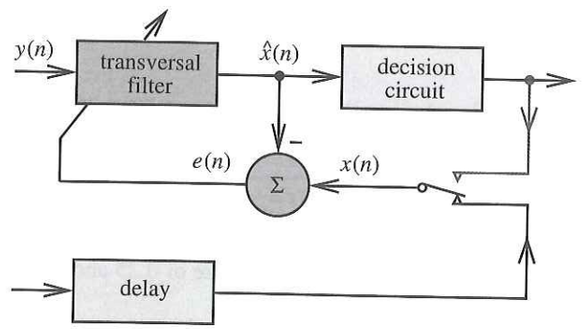


Figure 8.20
Decision directed equaliser design.